

Marcadores personalizados

Guía completa para añadir iconos, carteles informativos y ventanas emergentes custom en Google Maps.

1 · Icono del marcador

Icono desde drawable [drawable]

Reemplaza el pin rojo por cualquier imagen en res/drawable.

```
val icono = BitmapDescriptorFactory
    .fromResource(R.drawable.mi_icono)

mMap.addMarker(
    MarkerOptions()
        .position(madrid)
        .title("Madrid")
        .icon(icono)
    )
```

Cambiar color del pin [color]

Misma forma de pin, solo cambia el color usando las constantes HUE_*.

```
mMap.addMarker(
    MarkerOptions()
        .position(madrid)
        .title("Madrid")
        .icon(
            BitmapDescriptorFactory
                .defaultMarker(
                    BitmapDescriptorFactory.HUE_BLUE
                )
        )
    )
```

Icono desde Bitmap (escalar imagen) [\[bitmap\]](#)

Útil para redimensionar el icono antes de aplicarlo al marcador.

```
fun bitmapFromDrawable(context: Context, resId: Int, width: Int, height: Int):  
    BitmapDescriptor {  
    val drawable = ContextCompat.getDrawable(context, resId)!!  
    val bmp = Bitmap.createBitmap(width, height, Bitmap.Config.ARGB_8888)  
    val canvas = Canvas(bmp)  
    drawable.setBounds(0, 0, width, height)  
    drawable.draw(canvas)  
    return BitmapDescriptorFactory.fromBitmap(bmp)  
}  
  
// Uso:  
val icono = bitmapFromDrawable(requireContext(), R.drawable.mi_icono, 80, 80)  
mMap.addMarker(MarkerOptions().position(madrid).icon(icono))
```

2 · Snippet — subtítulo en la burbuja

Snippet básico [\[snippet\]](#)

Al tocar el marcador aparece una burbuja nativa con título y subtítulo. Sin código adicional, funciona directo.

```
mMap.addMarker(  
    MarkerOptions()  
        .position(madrid)  
        .title("Madrid")  
        .snippet("Capital de España\nPoblación: 3.3M")  
)
```

El snippet acepta saltos de línea con \n pero no soporta HTML ni estilos. Para eso, usa InfoWindowAdapter.

Escuchar click en la burbuja [\[listener\]](#)

Detecta cuando el usuario toca la ventana emergente para abrir un detalle, navegar, etc.

```
mMap.setOnInfoWindowClickListener { marker ->
    val destino = marker.position
    Toast.makeText(
        requireContext(),
        "Abriendo: ${marker.title}",
        Toast.LENGTH_SHORT
    ).show()
    // Aquí puedes navegar, abrir un fragment, etc.
}
```

3 · InfoWindow personalizado

Layout XML del InfoWindow [\[res/layout\]](#)

Crea el archivo `res/layout/custom_info_window.xml`:

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:background="@drawable/rounded_bg"
    android:padding="8dp">

    <ImageView
        android:id="@+id/infoImagen"
        android:layout_width="150dp"
        android:layout_height="100dp"
        android:scaleType="centerCrop"/>

    <TextView
        android:id="@+id/infoTitulo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textStyle="bold"/>

    <TextView
        android:id="@+id/infoDescripcion"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

InfoWindowAdapter en Kotlin [\[adapter\]](#)

Implementa el adaptador en tu `onMapReady`:

```
mMap.setInfoWindowAdapter(object : GoogleMap.InfoWindowAdapter {

    override fun getInfoWindow(marker: Marker): View? {
        return null // null = fondo estándar + nuestro contenido
    }

    override fun getInfoContents(marker: Marker): View {
        val view = layoutInflater.inflate(R.layout.custom_info_window, null)
        val datos = marker.tag as? Map<*, *>

        view.findViewById<TextView>(R.id.infoTitulo).text = marker.title
        view.findViewById<TextView>(R.id.infoDescripcion)
            .text = datos?.get("descripcion") as? String ?: marker.snippet

        val imgRes = datos?.get("imagen") as? Int
        if (imgRes != null) {
            view.findViewById<ImageView>(R.id.infoImagen).setImageResource(imgRes)
        }
        return view
    }
})
```

Importante: Las InfoWindow son snapshots estáticos, no vistas vivas. Si cargas imágenes con Glide o Picasso, debes llamar `marker.showInfoWindow()` cuando la imagen termine de cargar para refrescar el snapshot.

4 · Datos extra con `marker.tag`

Adjuntar datos al marcador [\[tag\]](#)

El campo `tag` permite guardar cualquier objeto en el marcador sin variables globales.

```
val marker = mMap.addMarker(
    MarkerOptions()
        .position(madrid)
        .title("Madrid")
)

marker?.tag = mapOf(
    "imagen" to R.drawable.madrid_foto,
    "descripcion" to "Ciudad fundada en el siglo IX",
    "poblacion" to 3_300_000,
    "url" to "https://madrid.es"
)
```

Leer tag al hacer click [\[onClick\]](#)

Accede a los datos del tag cuando el usuario toca el marcador.

```
mMap.setOnMarkerClickListener { marker ->
    val datos = marker.tag as? Map<*, *>
    val desc = datos?.get("descripcion") as? String

    Toast.makeText(requireContext(), desc, Toast.LENGTH_SHORT).show()

    false // false = comportamiento normal (muestra burbuja)
}
```

Data class — mejor práctica [\[data class\]](#)

En vez de un Map, usa una data class tipada para mayor seguridad en tiempo de compilación.

```
data class LugarInfo(
    val descripcion: String,
    val imagen: Int,
    val url: String
)

marker?.tag = LugarInfo(
    descripcion = "Fundada siglo IX",
    imagen = R.drawable.foto,
    url = "https://madrid.es"
)

// Lectura limpia y con tipos:
val info = marker.tag as? LugarInfo
```

5 · Tabla de referencia rápida

MÉTODO	PARA QUÉ SIRVE	COMPLEJIDAD
<code>.icon()</code>	Cambiar la imagen/color del pin	[baja]
<code>.title()</code>	Título en la burbuja al tocar	[baja]
<code>.snippet()</code>	Subtítulo simple bajo el título	[baja]
<code>marker.tag</code>	Guardar datos extra en el marcador	[media]
<code>setInfoWindowAdapter</code>	Burbuja completamente personalizada con imagen	[alta]

MÉTODO	PARA QUÉ SIRVE	COMPLEJIDAD
<code>setOnInfoWindowClickListener</code>	Acción al tocar la burbuja emergente	[media]
<code>setOnMarkerClickListener</code>	Acción al tocar el pin directamente	[media]
<code>marker.showInfoWindow()</code>	Forzar refresco del InfoWindow tras cargar imagen async	[media]

■ **Recuerda:** Los `InfoWindow` son snapshots — una vez renderizados no se actualizan solos. Si cargas imágenes asíncronas (Glide, Picasso, Coil), llama `marker.showInfoWindow()` en el callback de la imagen para forzar el redibujado.